CoT Loop Detector Prefill Follow-up
Detailed Summary of Reopened Thread Rounds 1–12

2026-03-13 21:39 UTC

Executive Summary

The reopened thread started from a real concern: the completion-side classifier was only a proof of concept, while the actual goal is *prefill-stage* classification, and a classifier with roughly 50% precision and 50% recall is not useful. The work in this thread therefore attacked the problem in three phases: first isolate whether the prefill signal survives stronger confound controls, then test whether representation or training changes can improve the controlled prefill detector, and finally change the *target* itself once the old one-bit label looked exhausted.

The main scientific conclusions are:

- The prefill detector is *not* pure noise. After controlling for source and prompt length, and after training against an explicit metadata baseline, hidden-state probes still beat metadata by a nontrivial margin.
- Much of the naive prefill lift was shortcut-driven. Prompt length was the dominant confound uncovered in the early rounds.
- Representation and objective tuning on the old one-bit target largely saturated. The strongest controlled prefill arm remained the Round 6 metadata-aware all-layer last-token anchor with matched PR-AUC about `0.6498`.
- The major bottleneck was the label itself. On the preserved prompt subsets, small-horizon onset targets were effectively absent: `loop_by_64` was all-negative, and a scout showed zero events through `1024` and only `1/256` by `2048`.
- Late-horizon onset is not absent, just late. A long-horizon scout found `39/256` positives by `16384` and `62/256` by `30000`, with median first hit at token `11757`.
- The next scientifically correct branch is therefore preserved-subset training on `loop_by_16384`, not an immediate composition shift. That round was prepared but not launched because the GPU node was returning devices already occupied by another user's external jobs.

Metric Guide and Jargon

Prefill

Activations computed while the model reads the prompt, before any new answer tokens are

generated.

Completion or posterior
Activations computed after generation begins. These are a useful upper bound, but they are not the target problem here.

Probe
A small classifier trained on frozen activations. In this project the probes were mainly linear models or multi-layer perceptrons (MLPs).

PR-AUC
Area under the precision-recall curve. This is usually the better headline metric when positives are rare, because accuracy and even ROC-AUC can look overly flattering under imbalance.

ROC-AUC
Area under the receiver-operating-characteristic curve. Useful, but not sufficient by itself under imbalance.

Macro-F1
The F1 score computed for each class and then averaged, so the majority negative class does not dominate the number.

Metadata baseline
A classifier that only sees cheap prompt-level metadata such as source and prompt length, not hidden states. Here the validated legacy baseline used source, prompt length, newline count, digit count, and LaTeX-dollar count.

Natural test
The ordinary preserved test distribution. In these rounds it was the `560`-example MATH-500 plus AIME test set with `115` positives.

Matched test
A cleaner evaluation slice where each positive is paired with a source-matched and near-length-matched negative, so prompt-length shortcuts are harder to exploit.

Source-control
A training subset with the same source and class counts as a stricter control arm, but without matching prompt lengths. This isolates source composition from prompt length.

Anchor
The main prefill representation used in the controlled rounds: the all-layer last-token summary at the prompt boundary.

`loop_by_H`
A binary label that is positive if the loop detector fires by generation horizon `H`. Example: `loop_by_16384` asks whether a loop appears by token `16384`.

Onset CDF
The cumulative fraction of prompts whose first loop event has appeared by each horizon. It answers "how early do loops start?" rather than only "do they ever happen?"

Survival scout

A one-pass rollout used to estimate first-hit timing and right-censoring. Right-censoring means some prompts still have no observed loop by the maximum decode length, so their true onset could be later than the scout limit.

Within source×length bins

Metrics computed inside groups that share source and coarse prompt-length range, then averaged. This is a harsher check against shortcut exploitation.

Baseline Before the Reopened Thread

The reopened loop did *not* start from zero. The relevant pre-thread baseline was Round G from the same project:

- Best prefill arm: all-layer prefill last-token concat with macro-F1 `0.6641 +/- 0.0172`, PR-AUC `0.4737 +/- 0.0042`, ROC-AUC `0.7520 +/- 0.0017`, and positive F1 `0.4955 +/- 0.0129`.
- Completion-side upper bound: completion-view probes were much stronger, with PR-AUC around `0.805` and macro-F1 around `0.80`.

That baseline framed the entire reopened thread. The question was not whether activations contain *any* signal. The question was whether *prefill-only* activations contain enough robust signal once prompt shortcuts are controlled, and if not, whether the right fix is the representation, the training objective, or the label.

Round-by-Round Spine

Round	Main question	What changed	Main result and decision
1	Is prefill lift mostly dataset composition?	Dropped GSM8K, then matched source and prompt length on the preserved prefill tensors.	Prefill signal dropped materially under source+length matching but stayed above metadata. Decision: decompose train size, source mix, and prompt length separately.
2	Which confound matters most: train size, source mix, or length?	Added size-control and source-control arms plus a matched evaluation slice.	Prompt length was the dominant shortcut. Decision: stop more broad composition sweeps and train against metadata directly.
3	Does hidden-state signal survive explicit metadata conditioning?	Fit metadata-only baseline first, then trained the hidden probe as an additive residual over the frozen metadata logit.	Yes. Controlled prefill signal remained clearly above metadata. Decision: hold this frame fixed and test better prefill summaries.
4	Can cheap prompt-wide summaries replace the anchor?	Rebuilt fixed splits with multiple new prompt summaries, no new rollouts.	No replacement beat the metadata-aware anchor. Decision: test whether those views help only when fused with the anchor.

Round	Main question	What changed	Main result and decision
5	Is boundary context complementary to the anchor?	Added fusion arms such as `anchor + last16`.	Modest local lift for `anchor + last16`. Decision: verify under a cleaner selector and capacity control.
6	Does the Round 5 gain survive a proper holdout selector?	Moved selection to a train-side holdout and added a low-rank fusion control.	No. The anchor became best again. Decision: test whether the bottleneck is objective rather than representation fusion.
7	Can length-robust losses rescue the anchor?	Tried larger-batch BCE, equal-bin BCE, and equal-bin BCE plus within-bin ranking.	No objective tweak beat the Round 6 anchor. Decision: stop tuning the same one-bit anchor classifier and change target or representation.
8	Is one last order-sensitive suffix representation worth trying before changing targets?	Added a strided tail-token suffix view and hybrid anchor+tail arm.	Suffix order helped locally, but no arm beat the earlier Round 6 anchor. Decision: move to richer supervision.
9	Does the smallest onset target work?	Added `loop_by_horizon` labels and rebuilt the preserved fixed subsets at `H=64`.	`loop_by_64` was all-negative on both preserved subsets. Decision: do not guess another horizon blindly; run a scout.
10	How early do loops actually start on the preserved subset?	Ran a `256`-prompt scout to `2048`.	Zero events through `1024`, only `1/256` by `2048`. Decision: small-horizon binaries are dead; run a long-horizon survival scout.
11	Is there enough late-onset mass to stay on the preserved subset?	Ran a `30000`-token survival scout on the same `256` prompts.	Yes. `39/256` by `16384`, `62/256` by `30000`. Decision: stay on preserved split and train `loop_by_16384`.
12	Can the selected late-horizon round be launched immediately?	Prepared the full preserved-subset rebuild and anchor linear-vs-MLP follow-up.	Scientifically ready, operationally blocked. Slurm kept allocating GPUs already occupied by another user's external jobs.

Detailed Chronology

Round 1: Composition Controls

Question. If the prefill classifier looks decent, is it because the hidden states truly encode loop risk, or because the probe is exploiting easy prompt-level structure such as source mix and prompt length?

What changed. I reused the preserved Round G prefill tensors, so no new rollouts were needed. First I measured how strong a metadata-only classifier already was on the same natural test set. Then I built two controlled training subsets:

1. a **math-only** subset that drops GSM8K but keeps competition-math and OpenR1-Math;
2. a **math-only source+length-matched** subset that pairs positives and negatives by source and prompt-length order.

Key numbers.

- Metadata-only baseline on the preserved natural test: ROC-AUC `0.6590`, PR-AUC `0.3360`.
- Math-only MLP: macro-F1 `0.6782 +/- 0.0086`, PR-AUC `0.4704 +/- 0.0302`, ROC-AUC `0.7482 +/- 0.0060`.
- Math-only source+length-matched MLP: macro-F1 `0.6335 +/- 0.0031`, PR-AUC `0.4092 +/- 0.0074`, ROC-AUC `0.7172 +/- 0.0043`.

Interpretation. The prefill detector was not just random guessing, but a substantial part of its apparent lift came from coarse prompt structure. Dropping GSM8K alone barely hurt the best prefill arm, which meant the effect was not a single-source artifact. Matching by source and prompt length, however, produced a clear drop. The remaining signal above metadata was real, but clearly smaller than the raw Round G number suggested.

Design decision. Do not spend the next round on larger probes over the same feature family. Instead, separate train-size loss, source reweighting, and prompt-length control more cleanly.

Round 2: Size, Source, and Length Decomposition

Question. Was the Round 1 loss mainly caused by fewer training examples, by source rebalance, or by prompt-length control?

What changed. I kept the preserved natural test set fixed, reused the same prefill tensors, and added two new train arms:

- **size-control**: same train size as the matched arm, sampled randomly;
- **source-control**: same train size and same per-source/per-class counts as the matched arm, but random over prompt lengths.

I also built a new **matched test** with `115` positives and `115` near-length negatives within source, so the evaluation itself was less shortcut-friendly.

Key numbers.

- Natural-test hidden PR-AUC: `0.4704` (math-only), `0.4579` (size-control), `0.4562` (source-control), `0.4092` (length-matched).
- Matched-test hidden PR-AUC: `0.7020` (math-only), `0.6605` (size-control), `0.6780` (source-control), `0.6211` (length-matched).
- Even the strict length-matched arm still beat metadata on the matched test by about `+0.0829` PR-AUC and `+0.0968` ROC-AUC.

Interpretation. The dominant confound was prompt length. The drop from `math-only` to `size-control` was small, so train size was not the main story. The gap from `size-control` to `source-control` was also small, so source rebalance by itself was not the main story either. The real collapse happened from `source-control` to `length-matched`. This was the point where the thread stopped treating source and length as vague nuisances and started treating prompt length as the central shortcut.

Design decision. Hold the controlled subsets fixed and ask a sharper question: does hidden-state signal survive after *explicitly* conditioning on metadata, not just after changing dataset composition?

Round 3: Residual-over-Metadata Training

Question. Once a metadata model is already in the prediction, does a prefill hidden-state probe still add useful information?

What changed. I added a dedicated trainer that first reconstructs prompt metadata from preserved sample IDs, fits a metadata-only model, freezes its logit, and then trains the hidden-state probe as an *additive residual* on top of that logit. A smoke run showed that the older "legacy" metadata baseline reproduced Round 2 cleanly, while a supposedly stronger interaction-rich baseline actually underperformed, so I kept the validated legacy control instead of overclaiming.

Key numbers.

- Legacy metadata baseline: natural PR-AUC `0.3346`, ROC-AUC `0.6567`; matched PR-AUC `0.5487`, ROC-AUC `0.5408`.
- Metadata-residual hidden probe on `sourcectrl` training: natural PR-AUC `0.4201 +/- 0.0069`, ROC-AUC `0.7504 +/- 0.0014`; matched PR-AUC `0.6474 +/- 0.0088`, ROC-AUC `0.6857 +/- 0.0073`.
- Incremental lift over metadata: `+0.0855` natural PR-AUC and `+0.0987` matched PR-AUC.

Interpretation. This was the first round that made the "prefill signal is real" claim substantially harder to dismiss. The hidden-state branch was no longer benefiting merely from a cleaner subset; it had to add log-odds on top of a frozen metadata predictor. It still did. That meant the project had a defensible controlled prefill signal, even if it remained much weaker than completion-side classification.

Design decision. Keep the metadata-aware frame fixed. The next lever should be better prefill summaries, not more train/test composition churn.

Round 4: Prompt-Wide Summary Replacements

Question. Can a better prompt summary replace the all-layer last-token anchor?

What changed. I rebuilt prefill views on the exact preserved sample IDs and labels, again without new rollouts. New prompt summaries included last-16 pooling, short boundary deltas, late-vs-middle drift, and a small hybrid summary.

Key numbers.

- Anchor: natural PR-AUC `0.4182 +/- 0.0114`, matched PR-AUC `0.6416 +/- 0.0167`, matched within-bin PR-AUC `0.6494`.
- Best replacement (`boundary_delta8`): natural PR-AUC `0.4099`, matched PR-AUC `0.6253`.
- Worst replacement (`late_vs_middle_delta`): natural PR-AUC `0.3702`, matched PR-AUC `0.6092`.

Interpretation. Cheap prompt-wide summaries were not useless, but none of them beat the anchor as a replacement. The project therefore stopped viewing "simple pooling over more prompt positions" as the likely fix.

Design decision. Test whether those views are only helpful as augmentations *on top of* the anchor.

Round 5: Anchor Augmentation

Question. Is extra boundary context complementary when the anchor remains in the model?

What changed. I extended the residual trainer to concatenate multiple feature views while verifying sample-ID and label alignment, then trained three augmentation arms on the fixed Round 4 tensors.

Key numbers.

- Best arm, `anchor_plus_last16`: natural PR-AUC `0.4313 +/- 0.0030`, matched PR-AUC `0.6501 +/- 0.0020`, matched within-bin PR-AUC `0.6533`.
- Relative to the Round 4 anchor: `+0.0130` natural PR-AUC, `+0.0085` matched PR-AUC, `+0.0039` matched within-bin PR-AUC.
- Larger raw fusion was not automatically better: `anchor_plus_last16_delta8` lost ground on matched metrics.

Interpretation. Round 4's failure was specifically about *replacement*, not about the whole boundary-context family. There was a real but modest gain from adding a coarse last-16 summary on top of the anchor.

Design decision. Before accepting that gain, stress-test it under a proper train-side holdout selector and a capacity-controlled fusion arm.

Round 6: Holdout-Selected Capacity Diagnostic

Question. Does the Round 5 augmentation gain survive once checkpoint selection is moved off the evaluation set?

What changed. I added a fixed train-side holdout and a low-rank bottleneck option, then ran three arms under the same metadata-aware frame: `anchor`, `anchor_plus_last16`, and `anchor_plus_last16_lowrank256`

Key numbers.

- Anchor: natural PR-AUC `0.4138 +/- 0.0045`, matched PR-AUC `0.6498 +/- 0.0052`, matched within-bin PR-AUC `0.6641`.
- Raw `anchor + last16`: natural PR-AUC `0.4176 +/- 0.0014`, matched PR-AUC `0.6418 +/- 0.0025`, matched within-bin PR-AUC `0.6348`.
- Low-rank fusion: natural PR-AUC `0.4151 +/- 0.0054`, matched PR-AUC `0.6453 +/- 0.0061`, matched within-bin PR-AUC `0.6521`.

Interpretation. Under cleaner selection, the anchor became best again. The low-rank arm showed that the extra boundary view was not empty, but the apparent Round 5 gain was not robust enough to displace the anchor.

Design decision. Treat fusion as second-order. Next test whether the real bottleneck is the loss function rather than the representation.

Round 7: Objective and Selector Sweep

Question. Can length-robust objectives or within-bin ranking rescue the anchor without changing features?

What changed. I extended the trainer to support checkpoint selection on within-bin metrics, equal-bin binary cross-entropy (BCE), and an optional within-bin pairwise ranking auxiliary. I also recovered a "free" diagnostic by reselecting the saved Round 6 anchor logs using within-bin PR-AUC, so I could isolate selector effects without retraining.

Key numbers.

- Free selector-only reselection of the Round 6 anchor: matched PR-AUC fell from `0.6498` to `0.6254`; matched within-bin PR-AUC fell from `0.6641` to `0.6374`.
- Large-batch BCE control: matched PR-AUC `0.6381 +/- 0.0182`, matched within-bin PR-AUC `0.6496`.
- Equal-bin BCE: matched PR-AUC `0.6189 +/- 0.0058`, matched within-bin PR-AUC `0.6146`.
- Equal-bin BCE plus ranking: matched PR-AUC `0.6270 +/- 0.0135`, matched within-bin PR-AUC `0.6242`.

Interpretation. Nothing in this family beat the original Round 6 anchor. Selector alignment was not a free fix, equal-bin BCE overcorrected, and the ranking auxiliary only partly repaired the damage. This was the point where the thread concluded that small training-objective tweaks were not the dominant bottleneck.

Design decision. Stop tuning the same one-bit static-anchor classifier. The next round should change the target or, at most, try one last genuinely order-sensitive representation.

Round 8: Order-Sensitive Tail Representation

Question. Before paying the full cost of new labels, is one last suffix-order representation check worth doing?

What changed. I confirmed from the builder path that preserved prefill datasets do not store rollout trajectories, so target changes would require new label-build passes rather than cheap relabeling. Given that constraint, I ran one last fixed-label representation test: a `tail64_strided_concat` suffix view that concatenates final-layer prompt-token states at offsets `{64, 48, 32, 16, 8, 4, 2, 1}` plus a short-prompt availability mask. After fixing an extractor dispatch bug and a matched-split loader bug, the three-arm batch completed.

Key numbers.

- Rebuilt anchor: natural PR-AUC `0.3957`, matched PR-AUC `0.6121`, matched within-bin PR-AUC `0.6198`.
- `tail64_strided_final`: natural PR-AUC `0.4064`, matched PR-AUC `0.6175`, matched within-bin PR-AUC `0.6420`.
- `anchor_plus_tail64_lowrank256`: natural PR-AUC `0.4057`, matched PR-AUC `0.6253`, matched within-bin PR-AUC `0.6307`.

- External reference still stronger: reproduced Round 6 anchor matched PR-AUC `0.6498`, matched within-bin PR-AUC `0.6641`.

Interpretation. Preserving suffix order was a real local gain over the rebuilt Round 8 anchor, especially on within-bin metrics, so the view was not empty. But none of the Round 8 arms beat the earlier controlled Round 6 anchor. That made representation-only work on the old one-bit target look saturated.

Design decision. Move to *richer supervision*. The next round should change the target, starting with compact onset-horizon labels.

Round 9: First Target Change, `loop_by_64`

Question. Does the smallest meaningful onset label already give a trainable target on the preserved subsets?

What changed. I added a new label surface to the existing stack:

- `eventual_loop`: the legacy one-bit label;
- `loop_by_horizon`: positive if the first loop appears by horizon `H`.

I also added tooling to materialize fixed-split JSONL prompt subsets while preserving original sample IDs, so metadata-aware evaluation stayed aligned to the preserved pools. After a runtime-environment repair and a vLLM memory-envelope repair, the `loop_by_64` batch completed its dataset build.

Key numbers.

- Source-control train: `2096` negatives, `0` positives.
- Source-control test: `560` negatives, `0` positives.
- Matched train placeholder: `230` negatives, `0` positives.
- Matched test: `230` negatives, `0` positives.

Interpretation. This was not a weak training result. It was a stronger statement: on the preserved prompt subsets, `loop_by_64` simply does not occur under the current loop detector. The old plan of trying `64`, then `128`, then `256` as a blind sweep no longer made sense.

Design decision. Stop guessing horizons. Run a scout that measures actual onset prevalence directly.

Round 10: Short-Horizon Scout

Question. If `H=64` is empty, where does onset mass actually begin?

What changed. I wrote a scout helper that samples a source-by-length stratified slice, runs one rollout pass, records first-hit timing, and derives post-hoc prevalence for multiple horizons from the same run. I then ran it on a stratified `256`-prompt source-control-train slice out to `2048`.

Key numbers.

- `loop_by_64`: `0/256`

- loop_by_128: 0/256
- loop_by_256: 0/256
- loop_by_512: 0/256
- loop_by_1024: 0/256
- loop_by_2048: 1/256
- Only observed first-hit step: 1423

Interpretation. Compact onset binaries were not merely poor at 64; they were essentially absent all the way through 1024, and still too sparse at 2048 to justify another full binary rebuild.

Design decision. Shift from binary-onset thinking to a long-horizon survival-style scout. The new question was whether the preserved subset had *late* trainable onset mass or whether composition had to change entirely.

Round 11: Long-Horizon Survival Scout

Question. Is there enough late-onset mass on the preserved subset to stay on the same fixed prompts, or must the project switch to an onset-enriched composition?

What changed. I ran the same 256-prompt stratified preserved slice out to 30000 tokens with a lighter rollout scheduler, and I consulted Athena repeatedly to turn the result into an explicit branch rule instead of an ad hoc judgment. While the scout was running, I also audited the preserved fixed-subset manifests and confirmed a subtle but important fact: the preserved subset remained balanced on the legacy eventual_loop bit, yet collapsed to all-negative at loop_by_64. So the problem was specifically *late onset*, not generic imbalance.

Key numbers.

- loop_by_2048: 2/256 (0.78%)
- loop_by_4096: 3/256 (1.17%)
- loop_by_8192: 17/256 (6.64%)
- loop_by_16384: 39/256 (15.23%)
- loop_by_30000: 62/256 (24.22%)
- First-hit timing among positives: minimum 810, p25 7209, median 11757, p75 19552, p90 23223, maximum 28403.

Interpretation. This round resolved the main scientific fork. Onset was still essentially absent at small and medium horizons, but there *was* enough late-onset mass by 16384 to support a preserved-subset follow-up. The earliest viable late-horizon target was therefore loop_by_16384. Going all the way to 30000 would have pushed the label too deep into the right tail; staying at 8192 would still have been too sparse.

Expected full-subset counts if the scout prevalence carries through.

- Source-control train: about 319 positives.
- Source-control test: about 85 positives.

- Matched test: about `35` positives.

Design decision. Stay on the preserved split and train a coarse late-horizon target at `loop_by_16384`. Do *not* pivot immediately to an onset-enriched composition shift.

Round 12: Prepared but Resource-Blocked

Question. Can the selected `loop_by_16384` branch be executed immediately?

What changed. I prepared a follow-up batch that rebuilds the preserved source-control and matched datasets using `label_target=loop_by_horizon@16384`, then trains the anchor representation with a small linear-versus-MLP comparison under the same preserved labels.

What was ready.

- New launcher: `round12_h16384_target.sbatch`
- Safer rollout scheduler: `dp=4`, `max_num_seqs=8`, `max_num_batched_tokens=4096`, `VLLM_GPU_MEMORY_UTILIZATI`
- Scientific target fixed: preserve the same prompt subsets and train `loop_by_16384`

Why it did not launch. The GPU node never became safe enough. Slurm looked empty from its own queue view, but `nvidia-smi` still showed another user's large external Python jobs sitting on GPUs `2`, `4`, and `6`. A fresh `-gres=gpu:4` probe kept returning `2,3,4,6`, which overlaps those occupied devices. Launching the real round into that state would very likely have produced a CUDA out-of-memory failure rather than a scientific result.

Interpretation. The blocker at the end of the thread was operational, not scientific. The design question was settled; the scheduler environment was not.

What the Thread Established

1. **Controlled prefill signal exists.** After explicit source and prompt-length controls, and after direct conditioning on metadata, prefill hidden states still contain loop-relevant information.
2. **Shortcut pressure is real and large.** Prompt length was the dominant confound in the early rounds, and many seemingly promising gains weakened once the evaluation became more controlled.
3. **Round 6 is the best controlled old-target baseline.** The metadata-aware anchor with train-side holdout selection remains the strongest defensible prefill detector on the legacy label, with matched PR-AUC around `0.6498`.
4. **Minor representation and objective tweaks are largely exhausted on the old one-bit target.** Boundary-context fusion, low-rank fusion, equal-bin losses, and within-bin ranking all failed to produce a new best model.
5. **The real label problem is timing.** Small-horizon onset labels are almost absent on the preserved prompt sets, but late-horizon onset is not.
6. **The earliest viable next target is `loop_by_16384`.** That is the central output of the entire Round 9–11 sequence.

Recommended Immediate Next Step

If the node clears, submit the already prepared preserved-subset Round 12 batch:

- rebuild preserved `sourcectrl` and `matched` datasets at `loop_by_16384`;
- train the anchor view with a small linear-versus-MLP comparison;
- evaluate matched performance primarily with PR-AUC and within-bin metrics, not accuracy alone.

If the node remains contested, the next intervention should be infrastructural rather than scientific: reserve or isolate safe GPUs, move to a cleaner node, or otherwise avoid Slurm allocations that overlap with external non-Slurm jobs. The thread already answered the design question it was supposed to answer.

Bottom Line

The reopened loop did not prove that prefill-stage loop detection is solved. It did something more important: it separated a misleading short-horizon story from a viable late-horizon story. The prefill detector is weaker than the completion detector, and much of the naive prefill lift was shortcut-inflated. But after controlling for those shortcuts, the signal is still real, and the long-horizon scout shows that the preserved prompt subsets are not hopeless. They are simply *late*. The scientifically correct next experiment is therefore the preserved `loop_by_16384` classifier round, and the reason it is not already finished is GPU allocation safety, not lack of a coherent target.